

生成概率还不够：AI 代码助手到底该高亮什么？

来自 Stanford 与 Microsoft Research 的 NeurIPS 2022 研究：与其高亮模型「没把握」的 token，不如高亮人「最可能改动」的 token。

GitHub Copilot、Amazon CodeWhisperer、DeepMind 的 AlphaCode 这类 AI 结对编程工具，如今能直接在编辑器里给出代码补全。它们很好用，但并不完美：一条错误的建议如果被人顺手采纳，可能悄悄埋下一个 bug，甚至一处安全漏洞。要拦下这样的建议，程序员得先发现它，而这比想象中更难，因为即便是专家也容易陷入自动化偏见（automation bias），也就是过度信任一台看起来很自信的机器。

一个很自然的思路，是借鉴输入法里的拼写检查：把建议中最需要复核的地方高亮出来，提醒人多看一眼。但一段代码补全并不是一个决策，而是几百个小决策，每个 token 一个。那么，到底该高亮哪些 token？最直觉的答案，是高亮模型自己「最没把握」的部分，也就是生成概率（generation probability）最低的那些 token，理由是低概率的文本比较「非常规」，因此更可疑。这套策略此前已被提出，甚至已经内置在 OpenAI 的 Playground 界面里。而这篇论文追问了一个更尖锐的问题：这真的是该高亮的东西吗？

在一项预注册（preregistered）、混合方法的研究中，作者招募了 30 名程序员，把生成概率高亮和一个全新的「编辑模型」（edit model）正面对比。后者预测的是人最可能改动哪些 token。这个视角的转换正是全文的关键：与其问模型在哪里不确定，不如问人最可能在哪里动手。结果是，在任务耗时、编辑精准度和用户偏好三个维度上，edit model 全面胜出；而生成概率高亮反倒是所有方案里表现最差的一个，甚至不如完全不给高亮。

论文：<https://arxiv.org/abs/2302.07248>

预注册：<https://osf.io/tymah>

最直觉的信号，为什么可能是错的

高亮低概率 token 的逻辑很有吸引力。语言模型在生成每个 token 时都会给它一个概率，那些它「犹豫过」、概率很低的 token，看起来最可能出错。把它们标出来，程序员的视线自然会被引向这些高风险的位置，就像拼写检查在拼错的单词下画一条波浪线。

但问题在于：模型的不确定，和人真正需要改动的地方，并不是一回事。模型可能对某个 token 非常自信，而人出于自己的编码风格、变量命名或边界情况仍然想改它；反过来，模型可能对某个其实完全正确、根本不会被碰的 token 犹豫不决。程序员真正需要被高亮的，不是模型「没把握」的地方，而是他们「大概率会伸手去改」的地方。这正是这项研究要检验的假设。

同一个模型，三副面孔

为了单独考察「高亮信号」的作用，作者把其他一切都固定住。研究里的三个工具背后是同一个模型——OpenAI 的 Davinci-002 Codex，给出的补全内容完全一致，唯一的区别只在于呈现方式。Prediction-only 工具直接展示原始补全，不做任何高亮；Generation-probability 工具高亮 Codex 最没把握的 token，概率阈值设为 71%；Edit-model 工具则依据一个学到的编辑模型，高亮最可能被修改的 token，阈值设为 66%。

这个编辑模型是在一个预备阶段训练出来的，训练数据来自 9 名程序员：他们不断修改 Codex 的输出，直到任务被正确解决，从而告诉模型人到底会动哪些 token。尤为关键的是，两个阈值经过调校，使得每种条件下所有任务显示的高亮总数相等。这个设计很重要：它保证了对比的是「高亮了哪些 token」，而不是「高亮了多少个」，于是行为上的任何差异，都只能归因于信号本身的质量。

这 30 名参与者都是某美国大型科技公司里有经验的 Python 程序员，每人为约一小时的实验获得 50 美元报酬。每人要解决三个来自 LeetCode 「easy」难度的编程任务，每个任务上限 10 分钟，过程中可以运行自己的代码，并用提供的一组单元测试来调试。任务顺序，以及工具与任务的搭配，都做了随机化处理，以保证对比公平。

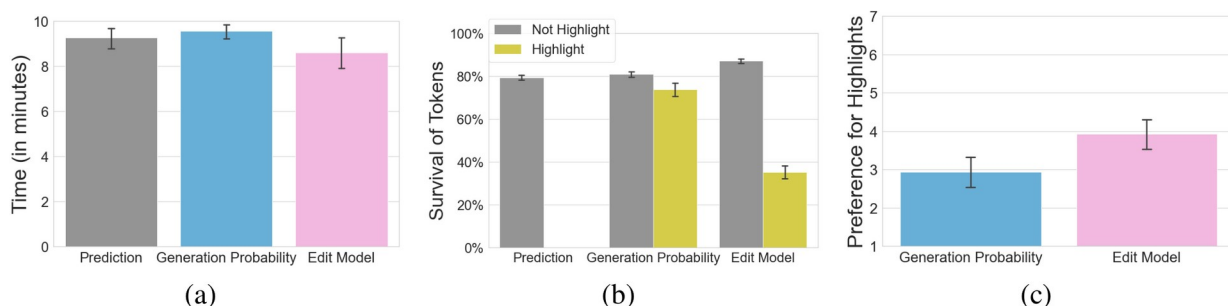


图 1 (a) 每个任务的耗时（上限 10 分钟）。(b) 各条件下 token 的「存活」比例，按 token 是否被高亮拆分。(c) 用户对高亮的主观偏好。Edit model 最快，它高亮的 token 存活率最低（说明高亮精准命中了真正会被改动的位置），也最受用户青睐。

高亮对了 token，人就更快

最醒目的结果关于速度（图 1a）。参与者在 edit model 高亮下完成任务最快，平均每个任务 8.59 分钟；在生成概率高亮下最慢，为 9.61 分钟；不给高亮的 Prediction-only 条件居中，为 9.27 分钟。两种高亮策略之间的差距高度显著（ $p = 0.003$ ）；而 edit model 相对于完全不高亮的优势，则是一个有希望但更弱的趋势（ $p = 0.06$ ）。

更值得玩味的是排名的另一端：生成概率高亮不仅输给了 edit model，还是三者中最慢的一个，甚至比 Prediction-only 还慢。高亮错了 token，不只是没帮上忙，反而似乎实实在在地拖慢了人——大概是因为它把注意力引向了并不需要关注的代码。换句话说，高亮「什么」，比「要不要」高亮更重要。

表 1 Edit model 高亮与生成概率高亮在三项核心指标上的对比。存活率指被高亮的 token 中未被改动的比例；数值越低，说明高亮越准确地命中了真正会被修改的位置。

指标	Generation Probability	Edit Model	p
任务耗时（分钟）	9.61	8.59	0.003
被高亮 token 的存活率	0.74	0.35	< 0.0001
偏好评分（7 点量表）	2.88	3.94	0.001

原因：编辑可能性，直接对准了人真正会改的地方

为了弄清 edit model 为何更好，作者追踪了哪些 token「存活」，也就是被参与者原封不动地保留（图 1b）。一个好的高亮，应该标出会被修改的 token，同时放过会被保留的 token。edit model 恰恰做到了这一点：它没有高亮的 token 存活率为 0.87，高于生成概率条件（0.81）和 Prediction-only 条件（0.79）；而它高亮的 token 存活率只有 0.35。生成概率则要浑浊得多：它高亮的 token 仍有 0.74 的存活率，和它没高亮的 token 几乎没差别。上述所有差异都极其显著（ $p < 0.0001$ ）。

说白了，edit model 的高亮能相当准确地预测人会改动什么，而生成概率的高亮和真实编辑之间只有微弱的对应关系。参与者对此也有切身感受（图 1c）：在一个综合了「高亮是否有用、是否愿意付费、是否让人分心」的 7 点量表上，edit model 得到 3.94 分，生成概率只有 2.88 分，差距同样显著（ $p = 0.001$ ）。

这意味着什么，又还有哪些悬而未决

结论简单却切中要害：要帮人复核 AI 生成的代码，应该高亮他们「大概率会改」的地方，而不是模型「恰好没把握」的地方。仅仅这一处改动，就让程序员更快、编辑更精准，也让他们明显更偏爱这款工具。生成概率——这个最容易从语言模型里直接读出、并且已经用在真实产品界面里的信号——还不够。

作者对局限也很坦诚。他们的编辑模型是「闭世界」（closed-world）的：它是在自己后来被评测的那批 AI 生成结果上训练出来的，属于近乎完美校准的最理想情形。能否学出一个「开放世界」、通用的编辑模型，以及它是否会同样改变人的行为，都还有待验证。此外，速度、精准度和偏好上的这些收益，能否转化为真正重要的高层结果——比如更准确的人类监督、更少的自动化偏见——同样尚未被证明。令人鼓舞的一点是：像 GitHub Copilot 这样的产品，本就把用户对建议的编辑记录当作一项性能指标来采集，而这种产品级、规模化的真实编辑数据流，恰恰正是训练一个开放世界编辑模型所需要的。